Lab 7 - Fuzz Testing

Mobile Data Plan Application

Hypothesis fuzz testing and corrected input validation

Evidence: original execution, minimized failures, corrected execution, passing tests

Final result: 12 tests passed; all three application functions fuzz-tested.

The original Lab 6 function behavior is preserved for valid inputs.

Invalid values now raise ValueError with explicit input rules.

The following pages contain execution evidence and the requested Python sources.

Original application execution

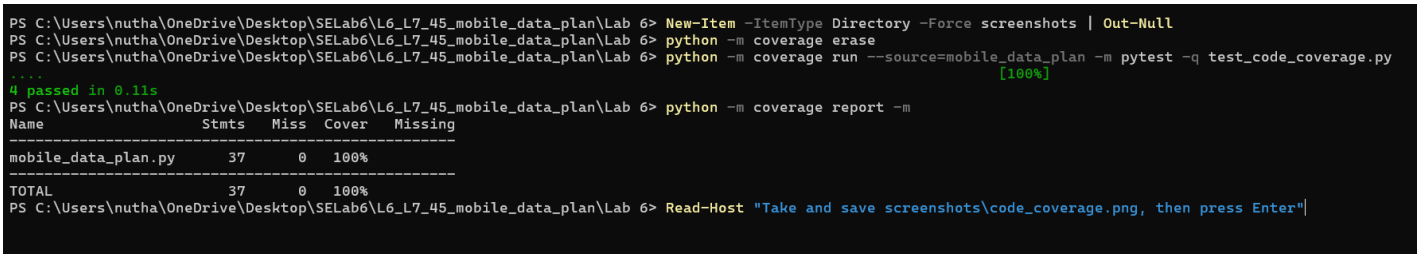
Direct calls against the Lab 6 module; Lab 6 coverage screenshots follow.

Original Lab 6 application - direct execution

```
$ python -c <three Lab 6 function examples>
overage_charge: 80
recharge_allowed: True
validity_status: grace
```

Original Lab 6 test execution

Existing Lab 6 screenshots showing the unmodified application test runs.



Original Lab 6 branch coverage

Existing branch-coverage execution evidence.



Failing Hypothesis examples

Minimized invalid inputs found against the original implementation.

Hypothesis failures before fixes

```
$ python -m pytest -v --hypothesis-show-statistics
test_overage_charge_rejects_invalid_usage - FAILED
  minimized invalid_data='' -> TypeError; False was also accepted
test_recharge_rejects_negative_amount - FAILED (amount=-1)
test_recharge_rejects_negative_pending_count - FAILED (pending_recharges=-1)
test_validity_rejects_boolean_day_counts - FAILED (days_left=False)
test_validity_rejects_fractional_day_counts - FAILED (days_left=0.5)
Result: 5 failed, 3 passed in 7.71s
```

Corrected application execution

Representative valid outputs from the corrected Lab 7 module.

Corrected Lab 7 application - direct execution

```
$ python -c <three corrected function examples>
overage_charge: 80
recharge_allowed: True
validity_status: grace
```

Final successful test execution

Hypothesis statistics show 100 examples for each applicable property.

Final Hypothesis test run

```
$ python -m pytest -v --hypothesis-show-statistics
collecting ... collected 12 items
test_processor.py::test_overage_charge_matches_plan_rate PASSED          [  8%]
test_processor.py::test_overage_charge_rejects_invalid_usage PASSED      [ 16%]
test_processor.py::test_overage_charge_rejects_non_boolean_flags PASSED  [ 25%]
test_processor.py::test_recharge_allowed_matches_limits PASSED           [ 33%]
test_processor.py::test_recharge_rejects_negative_amount PASSED          [ 41%]
test_processor.py::test_recharge_rejects_negative_pending_count PASSED   [ 50%]
test_processor.py::test_recharge_rejects_invalid_amount PASSED           [ 58%]
test_processor.py::test_recharge_rejects_non_integer_pending_count PASSED [ 66%]
test_processor.py::test_validity_status_matches_cutoffs PASSED           [ 75%]
test_processor.py::test_validity_rejects_boolean_day_counts PASSED       [ 83%]
test_processor.py::test_validity_rejects_fractional_day_counts PASSED    [ 91%]
test_processor.py::test_validity_rejects_non_boolean_flags PASSED        [100%]
============================== 12 passed in 1.20s ==============================
```

processor.py - page 1

```
import math
from numbers import Real


INCLUDED_GB = {"basic": 2, "standard": 5, "premium": 10}


def _is_finite_number(value):
    return (
        isinstance(value, Real)
        and not isinstance(value, bool)
        and math.isfinite(value)
    )


def overage_charge(plan, data_gb, is_roaming, is_loyal):
    if not isinstance(plan, str) or plan not in INCLUDED_GB:
        raise ValueError("unknown plan")
    if not _is_finite_number(data_gb) or data_gb < 0:
        raise ValueError("data usage must be a finite non-negative number")
    if type(is_roaming) is not bool or type(is_loyal) is not bool:
        raise ValueError("roaming and loyalty flags must be booleans")

    included = INCLUDED_GB[plan]
    if is_loyal and plan != "premium":
        included += 1
    extra = data_gb - included
    if extra <= 0:
        return 0
    if is_roaming:
        rate = 40
    else:
        rate = 15
    if extra > 10:
        rate = rate + 10
        if is_roaming:
            rate += 5
    return extra * rate


def recharge_allowed(amount, pending_recharges, is_kyc_pending):
    if not _is_finite_number(amount) or amount < 0:
        raise ValueError("amount must be a finite non-negative number")
    if type(pending_recharges) is not int or pending_recharges < 0:
        raise ValueError("pending recharges must be a non-negative integer")
    if type(is_kyc_pending) is not bool:
        raise ValueError("KYC pending flag must be a boolean")

    if is_kyc_pending:
        return amount <= 200
    if amount > 2000:
        return False
    return pending_recharges <= 2


def validity_status(days_left, is_prepaid, has_addon):
    if type(days_left) is not int:
        raise ValueError("days left must be an integer")
    if type(is_prepaid) is not bool or type(has_addon) is not bool:
        raise ValueError("prepaid and add-on flags must be booleans")

    if is_prepaid:
        cutoff = 0
```

processor.py - page 2

```
    else:
        cutoff = 3
    if has_addon:
        cutoff += 2
    if days_left >= cutoff:
        return "active"
    elif days_left >= cutoff - 15:
        return "grace"
    return "expired"
```

test_processor.py - page 1

```
import pytest
from hypothesis import given, strategies as st

from processor import overage_charge, recharge_allowed, validity_status


PLANS = {"basic": 2, "standard": 5, "premium": 10}


@given(
    plan=st.sampled_from(tuple(PLANS)),
    data_gb=st.floats(min_value=0, max_value=100, allow_nan=False, allow_infinity=False),
    is_roaming=st.booleans(),
    is_loyal=st.booleans(),
)
def test_overage_charge_matches_plan_rate(plan, data_gb, is_roaming, is_loyal):
    included = PLANS[plan] + int(is_loyal and plan != "premium")
    extra = max(0, data_gb - included)
    rate = 40 if is_roaming else 15
    if extra > 10:
        rate += 10 + (5 if is_roaming else 0)
    expected = 0 if extra == 0 else extra * rate
    assert overage_charge(plan, data_gb, is_roaming, is_loyal) == expected


@given(
    invalid_data=st.one_of(
        st.booleans(),
        st.text(),
        st.sampled_from((float("nan"), float("inf"), float("-inf"))),
    )
)
def test_overage_charge_rejects_invalid_usage(invalid_data):
    with pytest.raises(ValueError):
        overage_charge("basic", invalid_data, False, False)


@given(flag=st.one_of(st.integers(), st.text()))
def test_overage_charge_rejects_non_boolean_flags(flag):
    with pytest.raises(ValueError):
        overage_charge("basic", 1, flag, False)
    with pytest.raises(ValueError):
        overage_charge("basic", 1, False, flag)


@given(
    amount=st.integers(min_value=0, max_value=5000),
    pending_recharges=st.integers(min_value=0, max_value=10),
    is_kyc_pending=st.booleans(),
)
def test_recharge_allowed_matches_limits(amount, pending_recharges, is_kyc_pending):
    expected = (
        amount <= 200
        if is_kyc_pending
        else amount <= 2000 and pending_recharges <= 2
    )
    assert recharge_allowed(amount, pending_recharges, is_kyc_pending) is expected


@given(amount=st.integers(max_value=-1))
def test_recharge_rejects_negative_amount(amount):
    with pytest.raises(ValueError):
        recharge_allowed(amount, 0, False)
```

test_processor.py - page 2

```
064 |
065 |
066 | @given(pending_recharges=st.integers(max_value=-1))
067 | def test_recharge_rejects_negative_pending_count(pending_recharges):
068 |     with pytest.raises(ValueError):
069 |         recharge_allowed(100, pending_recharges, False)
070 |
071 |
072 | @given(
073 |     invalid_amount=st.one_of(
074 |         st.booleans(),
075 |         st.text(),
076 |         st.sampled_from((float("nan"), float("inf"), float("-inf"))),
077 |     )
078 | )
079 | def test_recharge_rejects_invalid_amount(invalid_amount):
080 |     with pytest.raises(ValueError):
081 |         recharge_allowed(invalid_amount, 0, False)
082 |
083 |
084 | @given(pending_recharges=st.one_of(st.booleans(), st.floats()))
085 | def test_recharge_rejects_non_integer_pending_count(pending_recharges):
086 |     with pytest.raises(ValueError):
087 |         recharge_allowed(100, pending_recharges, False)
088 |
089 |
090 | @given(
091 |     days_left=st.integers(min_value=-1000, max_value=1000),
092 |     is_prepaid=st.booleans(),
093 |     has_addon=st.booleans(),
094 | )
095 | def test_validity_status_matches_cutoffs(days_left, is_prepaid, has_addon):
096 |     cutoff = (0 if is_prepaid else 3) + (2 if has_addon else 0)
097 |     if days_left >= cutoff:
098 |         expected = "active"
099 |     elif days_left >= cutoff - 15:
100 |         expected = "grace"
101 |     else:
102 |         expected = "expired"
103 |     assert validity_status(days_left, is_prepaid, has_addon) == expected
104 |
105 |
106 | @given(days_left=st.booleans())
107 | def test_validity_rejects_boolean_day_counts(days_left):
108 |     with pytest.raises(ValueError):
109 |         validity_status(days_left, True, False)
110 |
111 |
112 | @given(days_left=st.floats(allow_nan=False, allow_infinity=False))
113 | def test_validity_rejects_fractional_day_counts(days_left):
114 |     if days_left.is_integer():
115 |         return
116 |     with pytest.raises(ValueError):
117 |         validity_status(days_left, True, False)
118 |
119 |
120 | @given(flag=st.one_of(st.integers(), st.text()))
121 | def test_validity_rejects_non_boolean_flags(flag):
122 |     with pytest.raises(ValueError):
123 |         validity_status(1, flag, False)
124 |     with pytest.raises(ValueError):
125 |         validity_status(1, True, flag)
```